# Week 2 Recap Paper: key

TRAINER | Cohort 2 | Week 2 | Saturday 17 October 2026 | 120 minutes | 33 items

**Marking**

1. Papers are swapped, so nobody checks their own.
2. The Academic TA reads the key out part by part, and the marker ticks or crosses each row of the answer sheet against the marking grid at the back of this key.
3. An item is right when its answer matches the key: every correct box and no other on a starred item, the letter on a word-bank or match item, the number on a work-it-out item (the working belongs to the discussion), and the whole sequence on an order item. The programme has set no partial-credit rule, so this key uses none.
4. The marker writes each part's ticks in its box beside the learner's rating, and their total as Items right, out of 33, then hands the paper back, so each learner reads their rating against their score part by part.
5. The TA collects the answer sheets and tallies the misses by part and by tag, never a ranking and never read out by name.
6. The TA enters every sheet in C2_W02_SAT_item_analysis_TRAINER.xlsx by seat, never by name: 1 for a tick, 0 for a cross, a blank for an item left empty, and each part's rating from step one. It orders the discussion from the most-missed item, flags any item to check, and sets each part's ratings beside its right rate.

## The blueprint

| Part | Title | Items | What it shows | Minutes | Easy | Medium | Hard |
|---|---|---|---|---|---|---|---|
| 1 | Can the warehouse give Anand Monday numbers that his analyst can audit line by line? | Q1 to Q6 (6) | whether you can write the Monday queries so every ratio, average and sample counts what it says | 16 | 2 | 2 | 2 |
| 2 | Can Anand sign what Kalpa collected against what it booked, with nothing lost or counted twice? | Q7 to Q11 (5) | whether you can join payments to orders without losing or repeating one, and hold what fails a check | 19.5 | 0 | 1 | 4 |
| 3 | Which members should Marketing protect and call, and did July to September keep pace with the plan? | Q12 to Q16 (5) | whether you can rank and flag members inside each segment and read a running total against its plan | 17 | 0 | 2 | 3 |
| 4 | What must Monday's customer table check before the growth team acts on it? | Q17 to Q20 (4) | whether you can build and check a table of one row per customer through merges and pivots in pandas | 16 | 0 | 0 | 4 |
| 5 | Which numbers in Monday's workbook can a director trust, and where did three public cases go wrong? | Q21 to Q27 (7) | whether you can build the workbook a director changes in the room and catch a wrong range, formula or base in a number | 22 | 0 | 4 | 3 |
| 6 | Can a food-delivery company's AI team trust the numbers behind its model and budget decisions? | Q28 to Q33 (6) | whether you can read an AI team's tables and code and say which decision a wrong number would change | 22.5 | 0 | 1 | 5 |
| | **Total** | **33** | | **113** | **2** | **10** | **21** |

### What guessing alone would score

A learner who guessed every item blind would average 6.4 of 33, since a written answer cannot be guessed from a list, and fewer than one guesser in twenty would reach 11. A score of 10 or below is therefore within reach of guessing alone, and the tally reads such a paper as a conversation to have on Monday, never as a result.

### Reading the items after marking

The workbook flags an item to check when fewer than one learner in five got it right, or when the bottom third of the room got it right more often than the top third. Both are this programme's own working rule for a room of 35. A flagged item is discussed as usual; the TA also sends it, with the room's rate, to the tracker's owner, because the fault may sit in the item rather than in the learners.

## The key

| Q | Key | Type | Part | Level | Tag | Day | Source |
|---|---|---|---|---|---|---|---|
| 1 | It prints 2 and 1. The note should say orders per member fell about 22 percent, from 2.36 to 1.84. | Applied maths | 1 | Hard | [F] | Mon | new |
| 2 | D (guarantee) | Fill in the blank | 1 | Easy | [F] | Mon | bank 2 |
| 3 | B (CTE) | Fill in the blank | 1 | Easy | [S] | Mon | bank 3 |

| Q | Key | Type | Part | Level | Tag | Day | Source |
|---|---|---|---|---|---|---|---|
| 4 | C | One correct option | 1 | Medium | [F] | Mon | new |
| 5 | A | One correct option | 1 | Medium | [D] | Mon | new |
| 6 | D | One correct option | 1 | Hard | [D] | Mon | new |
| 7 | A | Scenario set | 2 | Hard | [F] | Tue | new |
| 8 | E, F, B, C, A | Order the steps | 2 | Hard | [D] | Tue | new |
| 9 | B | One correct option | 2 | Hard | [S] | Tue | new |
| 10 | C | One correct option | 2 | Medium | [D] | Tue | new |
| 11 | C, E | More than one correct | 2 | Hard | [D] | Tue | new |
| 12 | D | One correct option | 3 | Hard | [D] | Wed | new |
| 13 | B | One correct option | 3 | Medium | [S] | Wed | bank 27 |
| 14 | A | One correct option | 3 | Medium | [S] | Wed | new |
| 15 | B | One correct option | 3 | Hard | [F] | Wed | new |
| 16 | C | One correct option | 3 | Hard | [D] | Wed | new |
| 17 | A | Scenario set | 4 | Hard | [F] | Thu | new |
| 18 | D | Scenario set | 4 | Hard | [D] | Thu | new |
| 19 | B | One correct option | 4 | Hard | [S] | Thu | new |
| 20 | B | One correct option | 4 | Hard | [D] | Thu | new |
| 21 | D (XLOOKUP with its if_not_found argument set) | Match the following | 5 | Medium | [F] | Fri | new |
| 22 | A (SUBTOTAL with function number 109) | Match the following | 5 | Medium | [F] | Fri | new |
| 23 | F (a first-row flag per order, then SUMIFS) | Match the following | 5 | Medium | [F] | Fri | new |
| 24 | C (a labelled input cell feeding a scenario line) | Match the following | 5 | Medium | [D] | Fri | new |
| 25 | The sheet reports -0.5 against 1.5, a gap of 2 points; B8 stops at row 5, and over all six countries both columns average 1.5, so the gap disappears. | Applied maths | 5 | Hard | [F] | Fri | new |
| 26 | Column D halves every change, 0.130 against 0.261, so the value at risk should read 140 million dollars, 20 million over the limit. | Applied maths | 5 | Hard | [D] | Fri | new |
| 27 | 2.00 dollars, about 8.5 percent of the 23.50 dollars charged | Applied maths | 5 | Hard | [F] | Fri | new |
| 28 | C | One correct option | 6 | Hard | [F] | Thu | new |
| 29 | A | One correct option | 6 | Hard | [S] | Wed | new |
| 30 | D | One correct option | 6 | Hard | [S] | Mon | new |
| 31 | B | One correct option | 6 | Hard | [F] | Tue | new |
| 32 | A | One correct option | 6 | Hard | [F] | Wed | new |
| 33 | D | One correct option | 6 | Medium | [F] | Mon | new |

# Why each answer holds

**Q1 Key It prints 2 and 1. The note should say orders per member fell about 22 percent, from 2.36 to 1.84.** Applied maths, Hard, [F], Mon

**Why it holds.** count(*) and count(DISTINCT customer_id) are both whole numbers, so Postgres divides them as integers and drops the fraction: 215 / 91 prints 2 for April to June, and 140 / 76 prints 1 for July to September. Orders per member were 2.36 and 1.84, and 1.84 against 2.36 is a fall of 22.0 percent. Accept 21 to 22 percent, about a fifth, or the fall from 2.36 to 1.84 given without a percentage, since each says what orders per member did. A note that quotes the printed 2 and 1 says orders per member halved, which is Monday's chapter 3 trap, and earns no tick. The fix is count(*)::numeric, and the check is to multiply back: 1 times 76 members is 76, where the orders are 140.

**In the interview.** Orders per member is a ratio of two counts, and Postgres divides two integers as integers, so I cast one side to numeric and multiply back by the denominator before the number leaves.

Anchor: A stakeholder's analyst must audit your query; what changes in how you write it, and what would you refuse to compute in a notebook?

**Q2 Key D (guarantee)** Fill in the blank, Easy, [F], Mon

**Why it holds.** A table has no stored order, so without ORDER BY the database returns whichever rows its plan reaches first, and two runs can differ. In Monday's chapter 6, a sample of five delivered app orders from July to September summed to Rs 3,900, and after a reload that rewrote two rows with the values they already held, the analyst's rerun drew a different five worth Rs 4,590. Every word in the bank is a noun and fits the blank's grammar, and only guarantee fits its sense. Sort is the near miss: a plan that reads an index may return sorted rows on some runs, which is why a sample can look stable until a reload, but the database promises nothing about it.

**In the interview.** LIMIT without ORDER BY returns whichever rows the database reaches first, so I order on a unique key before I limit.

Anchor: What does LIMIT without ORDER BY return?

**Q3 Key B (CTE)** Fill in the blank, Easy, [S], Mon

**Why it holds.** WITH names a common table expression, or CTE, which Monday's chapter 4 called a named step: the rest of the query reads it like a table. A subquery is the near miss, since it sits inline with no name. A view is named too, but CREATE VIEW stores it in the warehouse, and WITH never introduces one.

**In the interview.** A CTE names each step of the logic, so an auditor reads the query top to bottom, one decision per block.

Anchor: Explain the logical order in which a SQL query executes.

**Q4** **Key C** One correct option, Medium, [F], Mon

**Why it holds.** A member with no order in the quarter keeps one row from the LEFT JOIN, with NULL in every order column, so sum(o.amount) returns NULL for 44 of the 120 members. avg skips NULLs, so it divides Rs 4,13,380 by the 76 members who ordered and prints 5439. The head asked for the average over all 120 on the books, Rs 3,445, which needs coalesce(sum(o.amount), 0) in the member step. Monday's chapter 4 met the same skip from a CASE with no ELSE, and Kavya's rule from it is that an average names who is inside it.

**A.** avg reads NULLs without complaint and leaves them out; nothing in the query raises an error.

**B.** 3445 is the figure the head asked for, and the query prints it only once the member step turns a member with no order into 0 with coalesce; as written, those members carry NULL and avg skips them.

**D.** An aggregate skips NULLs; only arithmetic such as spend + NULL returns NULL.

**In the interview.** AVG skips NULLs, so an average over a LEFT JOIN is an average over the rows that matched unless I coalesce the missing ones to zero, and I say who is in every average.

Anchor: A stakeholder's analyst must audit your query; what changes in how you write it, and what would you refuse to compute in a notebook?

**Q5** **Key A** One correct option, Medium, [D], Mon

**Why it holds.** Anand's rule names the figures he wants every Monday, revenue, orders and customers by segment and channel, and asks for them from the warehouse with no notebooks and no exports. A weekly chart of orders per member for the head of Retail-Plus is none of those figures. It is exploration, and Monday's opening question settled where that goes: explore in a notebook, and report from the saved query. The notebook should read the warehouse, so it works on the same book as the suite. Friday's rule gives the direction a number travels, from the warehouse, through pandas for the analyst's iteration, to Excel for the room, and once the head decides what to report, that figure becomes a query in the suite.

**B.** Option b stretches the rule past its words. It covers the figures Anand files each Monday, and a suite that holds every exploratory chart stops being a list an analyst can audit.

**C.** An export ages from the day it lands and a workbook can be typed over, the two things Anand's rule forbids, and Excel is where a finished number meets the room.

**D.** Exploration is how a definition gets agreed; refusing to look leaves Finance to define the number blind.

**In the interview.** The reported number lives in the warehouse because it reruns unchanged on the source and every line can be audited; exploration lives in a notebook that reads the warehouse, and Excel presents what has been decided.

Anchor: Why would you compute a KPI in the warehouse rather than in a notebook?

### Q6 **Key D** One correct option, Hard, [D], Mon

**Why it holds.** The CASE has no ELSE, so it returns NULL for the two viewers under three seconds, and count skips NULLs: the calculated average divides all 30 seconds by 3 people, 10.0, where the defined one divides by all 5 who played, 6.0. Against the defined figure, 10.0 overstates 6.0 by 4.0 over 6.0, about 67 percent. TechCrunch (Devin Coldewey, 22 September 2016, read on 30 September 2026) quoted Facebook's definition and its calculation, put the inflation at 60 to 80 percent over two years and reported that billing was not affected.

**A.** 40 percent divides the gap by the wrong base, 4.0 over the calculated 10.0, and an overstatement is measured against the true figure, the defined 6.0.

**B.** Averaging the three long viewers alone gives (14 + 9 + 4) / 3 = 9.0 against 6.0, 50 percent, but the query's numerator keeps all the time watched, the short viewers' 3 seconds included.

**C.** count(CASE WHEN ... THEN 1 END) counts only the results that are not NULL, and a CASE with no ELSE returns NULL for the viewers it does not match, the same skip Monday's chapter 4 met inside avg.

**In the interview.** An average is defined by its denominator, and a filter or a CASE that shrinks the denominator raises the average with no change in the data and no error, so I print the numerator and the denominator beside the ratio.

Anchor: How do you present one number so it is not misread?

### Q7 **Key A** Scenario set, Hard, [F], Tue

**Why it holds.** A LEFT JOIN writes one row per matching payment row and keeps an order with none as one row of NULLs: 216 + 2 times 188 + 2 times 28 + 30 = 678 rows. count(p.payment_id) skips the 30 unpaid orders' NULLs where count(*) counts them, so it reads 678 minus 30 = 648, which is also what an INNER JOIN would return. The gateway's retried rows join twice like any other rows, and the eight payments with no order never meet an order. On Tuesday the same 678 rows, each carrying its order's booked amount, summed to Rs 19,29,04,410 of collected against Rs 9,84,00,000 booked, chapter 2's fan-out.

**B.** 462 and 432 read the join as one row per order and one payment row per paid order, but a join repeats an order once for each matching row, the fan-out of Tuesday's chapter 2.

**C.** Keeping each retry's two rows once gives 650 and 620, but a join matches rows by key and ignores what else they carry, so both rows repeat the order.

**D.** The eight payments with no order are counted in 686 and 656 as if they had joined, but a LEFT JOIN from orders keeps only rows that start from an order, and only a FULL OUTER JOIN brings in the payments whose order is missing.

**In the interview.** INNER keeps only matched rows and LEFT keeps every order with NULLs where nothing matched; both repeat an order once per matching payment row, so I size the join, rows out against orders in, before any rupee is summed.

Anchor: Your LEFT join grew the row count and revenue doubled; name the cause and the check.

**Q8** **Key E, F, B, C, A** Order the steps, Hard, [D], Tue

**Why it holds.** The step to leave out is d: 188 of the orders with two payment rows are instalments 1 and 2 of one invoice, and dropping them throws away real cash along with the 28 retries. On the same book, HAVING count(*) > 1 by order lists 216 orders, the 188 instalment orders and the 28 retries together, which is Tuesday's chapter 4 trap, and grouping by order and instalment lists the 28 retries alone. The retries go first (e), because a sum taken before them keeps the Rs 20,750 the gateway posted twice, which is posted and never collected. The payment rows then go to one collected figure per order (f), which is what the join (b) reads, so the result keeps one row per order. The checks (c) come before anything is sent (a), and on Kalpa's book they read 462 rows and a gap of Rs 17,54,930, the unpaid orders' booked value exactly.

**In the interview.** De-duplicate at the grain that defines a repeat, aggregate to the join's grain, join, prove the row count and the gap, then send.

Anchor: Design the validation you run before a joined number reaches Finance, and say what you do when it fails at the end of reporting day.

**Q9** **Key B** One correct option, Hard, [S], Tue

**Why it holds.** O-1's two instalments both fall inside the quarter, so the join gives O-1 two rows and sum(o.amount) counts its Rs 1,200 twice. O-2's only payment is dated 1 October, so its joined row fails the WHERE. O-3 has no payment, so its paid_date is NULL, BETWEEN on a NULL is unknown, and WHERE drops that row too. Two rows remain, booked Rs 2,400, against three orders worth Rs 2,500: the report lost two orders and doubled the one it kept.

**A.** 1 row assumes one row per order, which holds only once payments are brought to one row per order; O-1 has two payment rows inside the quarter, so the join repeats it.

**C.** O-3 stays in 3 rows only if its NULL date passed the filter, but a comparison with NULL is unknown and WHERE keeps only rows where it is true, which is how the quarter's dates in WHERE emptied the unpaid list in Tuesday's chapter 4.

**D.** Read as if it sat in the ON clause, the date filter would keep every order, with NULL payments on O-2 and O-3, for 4 rows; in WHERE it runs after the join and turns the LEFT JOIN into an INNER one, Tuesday's chapter 4 trap.

**In the interview.** A filter on the right-hand table of a LEFT JOIN belongs in ON, because WHERE runs after the join, drops the NULL rows and turns the LEFT JOIN into an INNER one; and the payments go to one row per order before the join.

Anchor: INNER against LEFT join: what does each drop or keep?

**Q10** **Key C** One correct option, Medium, [D], Tue

**Why it holds.** The gaps are app Rs 8,120 and store Rs 9,57,810, each equal to its unpaid list to the rupee, and web Rs 8,10,750, which exceeds its list's Rs 7,89,000 by Rs 21,750 that nobody has explained. Booked matches its source, so it goes for every channel; app's and store's collected reconcile, so they go; web's collected waits, and the open line names the failed check, the Rs 21,750 it leaves unexplained and when it will close. That is Tuesday's chapter 6 rule for a late failure, applied to each channel's figure on the page. The figures are illustrative: on Kalpa's own book, web's gap equals its unpaid list at Rs 7,89,000.

**A.** Adding the unexplained Rs 21,750 to web's unpaid list forces the bridge closed and puts on the list an amount that no unpaid order carries.

**B.** Store has the largest gap, and its gap equals its unpaid list to the rupee; the channel whose bridge fails is web, by Rs 21,750.

**D.** Holding all collected back keeps two figures that reconcile to the rupee, which the rule sends, so Anand loses app's and store's collected over a doubt that concerns web alone.

**In the interview.** Before a joined number reaches Finance I tie every figure back to one table alone: orders and booked against the orders table, the gap against booked less collected and against the unpaid list, and collected plus what was posted twice against what the feed posted. When one fails at the end of reporting day, booked leaves, a collected figure that does not reconcile is held, and the open line names the failed check, what it means and when it will close.

Anchor: Design the validation you run before a joined number reaches Finance, and say what you do when it fails at the end of reporting day.

**Q11** **Key C, E** More than one correct, Hard, [D], Tue

**Why it holds.** Lab B's file lost 5,365 rows in the conversion, 70,900 in the CSV against 65,535 in the sheet. Comparing rows loaded with the rows in the lab's own CSV sees the loss, and a sheet at exactly 65,535 data rows sits at the format's limit, which a check can treat as a stop. Rows loaded and rows in the sheet were both counted after the cut, so they agree, and Lab B loaded more rows than the day before, 65,535 against 61,020, so neither of those checks fires. Tuesday's chapter 3 met the same case: the loss sat in rows that never loaded, which a count of rows sent against rows loaded, taken back to the first count in the chain, measures. The facts are from the GOV.UK statement of 4 October 2020 (15,841 cases between 25 September and 2 October; files that exceeded the maximum file size) and The Register of 5 October 2020 (lab CSV files stored in the older .xls format, 65,536 rows a sheet, records past the limit left off and not counted), both read on 30 September 2026, and the three labs' counts are illustrative.

**A.** Rows loaded and data rows in the sheet were both counted after the conversion cut the file, so they agree at 65,535 and the check passes.

**B.** De-duplication removes rows; it cannot find rows that were never loaded.

**D.** Lab B loaded 65,535 rows against 61,020 the day before, so a check for a fall stays quiet while 5,365 rows go missing; a day-on-day check sees only a loss larger than the day's growth.

**In the interview.** Any step that can drop rows without a warning needs a count on both sides of it, taken back to the first count in the chain, and the load stops when they differ, since a larger limit only moves the point where rows are lost.

Anchor: Design the validation you run before a joined number reaches Finance, and say what you do when it fails at the end of reporting day.

**Q12** **Key D** One correct option, Hard, [D], Wed

**Why it holds.** RANK gives C-0185 and C-0242 the shared rank 50 and C-0259 rank 52, so 51 members hold a rank of 50 or better, and they are exactly the members who spent at least Rs 3,350, the fiftieth member's spend. DENSE_RANK leaves no gaps, so the tie at 48 (C-0189 and C-0206) lifts C-0185 and C-0242 to dense rank 49 and C-0259, on Rs 3,200, to 50, and its cut at 50 keeps 52 members, one of whom spent less than the fiftieth. ROW_NUMBER numbers every row once and drops C-0242, who spent as much as the fiftieth. RANK's 51 is the count each learner ran for the head's list in Wednesday's chapter 3, and the note to Marketing gives the rule, the count and the members at the line, the last place the list keeps, in one sentence.

**A.** DENSE_RANK shares ranks and then keeps C-0259, who spent Rs 3,200, less than both members at the fiftieth place, so its cut holds 52 members and the count of 51 belongs to RANK alone. It is the trap of Wednesday's chapter 3, where DENSE_RANK sent 52 Retail-Core members as the top fifty, two of whom tied with nobody.

**B.** ROW_NUMBER keeps exactly fifty by dropping C-0242, who spent what the fiftieth did, on an id nobody chose on purpose; if the budget demands fifty, the tiebreaker is a decision the head of Retail-Plus owns and states in advance.

**C.** Whole ties only leaves out C-0185 and C-0242, who both spent as much as the fiftieth, and ships the forty-nine the head refused.

**In the interview.** When the business says ties rank the same, I use RANK and state the list's length and its reason in the same sentence, because the list can ship more than N; if the budget demands exactly N, the tiebreaker becomes a decision someone owns.

Anchor: The business says 'ties rank the same'; which function, and how many rows might the top-N report ship?

**Q13** **Key B** One correct option, Medium, [S], Wed

**Why it holds.** A rank computed within PARTITION BY segment keeps every customer and starts again in each segment, and the outer query keeps the rows ranked three or better. The filter has to sit outside the window, because WHERE runs before any window is computed. The list in the stem is Wednesday's chapter 2 trap, the whole book numbered once and then split by segment: all 35 Business customers who ordered, since the smallest of their spends, Rs 2,25,000, is more than ten times the largest retail spend, Rs 21,740, then 11 Retail-Plus members, 4 Retail-Core customers and no Student.

**A.** GROUP BY collapses each segment to one row, so the customers are gone before LIMIT runs, and LIMIT applies once to the whole result, never per group.

**C.** One ORDER BY with LIMIT 3 is the stem's whole-book list cut shorter: it returns the top three overall, all Business, which repeats the first list's mistake.

**D.** HAVING keeps or drops whole groups by an aggregate, so it cannot choose rows inside a group.

**In the interview.** Top N per group is a window: rank inside PARTITION BY the group in a CTE, keep rank N or better outside it, and name the tie rule.

Anchor: Top-3 per group: GROUP BY or a window, and why?

**Q14** **Key A** One correct option, Medium, [S], Wed

**Why it holds.** OVER () puts the tier's total on every row without collapsing the rows, so each member's spend divides by Rs 4,13,380 and the shares add to 1. Grouping by member makes each sum the member's own spend, so every share reads 1. Dividing by avg(spend) OVER () compares each member with the average member, Rs 5,439. An ORDER BY inside the window turns the sum into a running total, the shape Wednesday's chapter 5 used against the plan line, so the top spender's share reads 1 and each later share divides by a larger part of the total.

**B.** GROUP BY customer_id makes sum(spend) each member's own spend, so every share reads 1.

**C.** An ORDER BY inside the window makes the sum a running total, so the top spender divides by their own spend and every share after it by a different part of the total.

**D.** Dividing by avg(spend) OVER () compares each member with the average member: a member at the average reads 1, and the results add to 76, the number of members, where shares add to 1.

**In the interview.** A share of the total is the row's value over sum() OVER (), a window with no partition and no order, which keeps every row and puts the total beside it.

Anchor: Top-3 per group: GROUP BY or a window, and why?

**Q15** **Key B** One correct option, Hard, [F], Wed

**Why it holds.** member_month has a row only for a month with an order, and LAG reads the previous row in the partition, never the previous calendar month. C-0161 and C-0171 fell from July to August and again from August to September, so both are flagged, and rightly. C-0185's September is compared with July (1,450 below 1,900) and July with June (1,900 below 2,690), and C-0216's September with July and July with May (2,540 below 4,300, below 6,440), so both are flagged too, although neither ordered in August: two of the four calls tell a member their spend fell in August and September when there was no August reading at all. A calendar check, that the two rows LAG reads are August and July, keeps the two genuine falls; it is Wednesday's chapter 6 fix, which cut sixteen calls to nine after C-0216 rang to say they had been travelling in August.

**A.** Reading LAG as a step back of one calendar month gives 2 calls, since C-0185 and C-0216 would meet an empty August and drop out; LAG returns whatever row came before in the window's order, and both are flagged.

**C.** 3 calls assume LAG(spend, 2) returns NULL for C-0216, but its rows are May, July and September, so on its September row LAG(spend, 2) returns May's 6,440. LAG returns NULL only on a member's first rows, and a comparison with that NULL flags nobody.

**D.** A member with a month missing before September stays in the flag, which the answer of 1 call forgets: LAG reads rows, so C-0161's empty May and June do not matter, and nor do the others' gaps.

**In the interview.** I build one row per customer per month, take LAG 1 and LAG 2 partitioned by customer and ordered by month, check that the two lagged rows are the two calendar months before, and flag a customer where each month is below the one before; a month with no order is no reading.

Anchor: How would you find customers whose spend fell two months in a row?

### Q16 **Key C** One correct option, Hard, [D], Wed

**Why it holds.** Booked to date less plan to date, read from the table at the seven readings, is -0.25, +1.95, +2.17, +1.57, +0.73, +0.79 and 0.00 crore; to the rupee, the lead was Rs 1,57,51,980 in the week of 17 August and Rs 79,70,130 in the week of 14 September. The quarter was behind after the first week, well ahead once the week of 13 July booked Rs 2.66 crore, furthest ahead in the week of 3 August at Rs 2,16,69,660, still about Rs 0.8 crore ahead in the week of 14 September, and level at the close, Rs 10 over a plan of Rs 9,83,99,990. The weekly rate did the giving back: six of the seven full weeks from 10 August booked below the plan's Rs 75,69,230 a week. A front-page sentence carries its period and its comparison, so a director reads the close and the path together. The figures are Wednesday's chapter 5 running total against plan_line, with the first plan week carrying the orders of 1 to 5 July.

**A.** The lead was Rs 1.95 crore in the week of 20 July and grew to Rs 2.17 crore by the week of 3 August, and in the week of 31 August it still stood at Rs 0.73 crore.

**B.** Option b reads the fortnights' bookings as the position: booked to date stayed above plan to date at every reading after the first week, although several weeks booked less than the plan's Rs 75,69,230.

**D.** The lead grew only until the week of 3 August, then fell from Rs 2.17 crore to Rs 0.73 crore by the week of 31 August.

**In the interview.** One number goes out with its period, its comparison and its base, and here the sentence carries the path as well as the close, because a quarter that ended level with plan after leading it by Rs 2.2 crore tells a director something the close alone hides.

Anchor: How do you present one number so it is not misread?

### Q17 **Key A** Scenario set, Hard, [F], Thu

**Why it holds.** A left merge keeps every customer and writes one row per matching feed row; the feed has 136 rows for 130 customers, so six of its rows repeat a customer already there, and each adds a row: 340 + 6 = 346 (six customers were sent twice, on 3 and 11 August). validate='one_to_one' checks that the key is unique on both sides and raises pandas' MergeError ("Merge keys are not unique in right dataset; not a one-to-one merge") before a merged table exists; an assert on the row count would stop the run too, once the table had been built. Unguarded, the pivot reads Rs 19,84,45,800, Rs 45,800 over the book. It is the merge the room ran on Kalpa's own feed in Thursday's chapter 2.

**B.** A left merge keeps each customer once only when the right key is unique; the six repeated rows add six rows, and the pivot reads Rs 45,800 over the book.

**C.** 352 counts each repeated customer's two rows as extra, where only the second is, and drop_duplicates() stops nothing, since the repeated rows differ in exposed_date and survive it.

**D.** An inner merge would keep 136 rows; a left merge keeps all 340 customers. indicator labels each row both or left_only, and stops nothing.

**In the interview.** A merge grows the left table only when a key repeats on the right; validate= states the relationship it must keep and raises MergeError the moment the keys break it, and a row-count check says the same thing in numbers.

Anchor: Which merge argument raises on duplicate keys, and which error?

### Q18 **Key D** Scenario set, Hard, [D], Thu

**Why it holds.** drop_duplicates with subset='customer_id' compares the customer id alone and, by its default keep='first', keeps each customer's first row in the order the file lists it. The tool lists each customer's 11 August send before its 3 August one, so C-0001 keeps 11 August, its second exposure, and the report credits the sale only from then on: orders placed from 3 to 10 August, after the real first exposure, go uncredited. On the week's book that is C-0001's order of 6 August, Rs 1,200. Thursday's chapter 2 applied the rule in order: sort the feed by exposed_date from the earliest, keep each customer's first row, then merge with validate='one_to_one'. Done here, that keeps 3 August for C-0001 and C-0002 and C-0012's one row, 130 rows in all, one per customer, and the merge returns 340 rows with spend on the book.

**A.** The 3 August send reads first as earliest, but the step keeps the first row in the order the file lists it, and the tool lists C-0001's 11 August send first.

**B.** Both sends would stay only if the step dropped rows that repeat in every column; subset='customer_id' compares the customer id alone, so one row per customer stays.

**C.** Neither send would stay only under keep=False, which drops every repeated customer; the default is keep='first', which keeps one row for each.

**In the interview.** A rule stated by date is applied by date: sort by exposed_date from the earliest, then keep each customer's first row, because a file's order is whatever the sender chose, and keep validate on, so a customer the feed repeats next week stops the run.

Anchor: A pivot's total disagrees with the warehouse; where do you look first?

### Q19 **Key B** One correct option, Hard, [S], Thu

**Why it holds.** pivot_table splits the orders by member and month, applies its default aggfunc, which is the mean, and combines one cell per group, so M1's two June orders print as their average, 2000.0, where M1 spent 4,000, and M2's June reads 1,200. melt turns the 2 by 2 view into 4 rows, one per member and month, including M2's empty July, and sum skips that NaN: 2,000 + 2,400 + 1,200 = 5600.0 against 8,800 of orders. The check is the grand total against the orders, and the fix is aggfunc='sum' with fill_value=0. On Kalpa's Retail-Plus months in Thursday's chapter 3, the default read a fall of 18 percent where the orders fell 29.4.

**A.** 4000.0 and 5 assume the pivot adds and that melt hands back the five orders, but the pivot averages unless aggfunc says otherwise, and melt returns one row per member and month.

**C.** A pivot that kept each cell's first order, and a melt that dropped the empty cell, would print 3000.0 and 3; the pivot averages, and melt keeps every member and month, NaN included.

**D.** 4000.0 3 8800.0 comes from a pivot that adds and a melt that drops the empty cell; the first of those is the misreading behind Thursday's chapter 3 trap, a fall of 18 percent read off a pivot that averaged.

**In the interview.** pivot_table widens a column's values into columns and averages unless told to sum; melt lengthens it back into rows; I write aggfunc= every time and tie the grand total to the source.

Anchor: pivot against melt: which widens and which lengthens?

**Q20** **Key B** One correct option, Hard, [D], Thu

**Why it holds.** The reference table holds one row per gene and panel, and TP53 sits on two panels, so the left merge writes TP53 twice: 5 rows from the lab's 4. '2-Sep' and '1-Mar' are SEPT2 and MARCH1 after Excel turned them into dates, so they match no symbol and keep a NaN panel: 3 rows carry a panel. Nothing raises. The lab's list goes back for the two symbols, fixed at their source, and the analyst decides at which grain a gene with two panels is counted before any total is taken. The finding is from Ziemann, Eren and El-Osta in Genome Biology, 2016, read on 30 September 2026.

**A.** 4 4 assumes pandas reads 2-Sep and 1-Mar back as gene symbols and that a left merge keeps one row per lab gene; the dates stay text that matches nothing, and TP53's two panels give it two rows.

**C.** An outer merge would add the reference rows nothing matched, SEPT2 and MARCH1, and print 7 5; a left merge keeps only the lab's rows and their matches.

**D.** Seeing that the two dates match nothing gives 4 2, which misses the fan-out: TP53 sits on two panels, so the left merge writes it twice.

**In the interview.** After every merge I check two things: the row count against the left table, because a key that repeats on the right fans the rows out, and the matches with indicator=True, because a key that changed on its way in never matches and never raises.

Anchor: SQL, pandas or Excel: how do you choose?

**Q21** **Key D (XLOOKUP with its if_not_found argument set)** Match the following, Medium, [F], Fri

**Why it holds.** XLOOKUP matches exactly by default, and its fourth argument, if_not_found, is what it shows for an id that is not in the list (Microsoft Support, XLOOKUP function, read on 29 September 2026). The unused technique beside it in the match table, VLOOKUP with its fourth argument left out, looks for an approximate match: in Friday's chapter 3 it returned C-0194's Rs 16,740, rank 15 on the list, for C-0195, a Retail-Plus member with no orders.

**In the interview.** A lookup has two exits, the row or a visible "not in the table", and I test it with an id I know is missing.

Anchor: A stakeholder wants to poke the numbers themselves; what do you give them and what do you never give them?

**Q22** **Key A (SUBTOTAL with function number 109)** Match the following, Medium, [F], Fri

**Why it holds.** SUBTOTAL with function number 109 adds only the rows on screen, while SUM adds the rows a filter hides: in Friday's chapter 6, filtered to Mumbai's 11 members, SUBTOTAL(109, ...) read Rs 1,56,790 and SUM still read the whole list's Rs 7,14,890, a budget 4.6 times too big.

**In the interview.** A total under a filtered list is SUBTOTAL(109), and I say in the cell's label what it adds.

Anchor: Your pivot shows a different total from the warehouse; where do you look first?

**Q23** **Key F (a first-row flag per order, then SUMIFS)** Match the following, Medium, [F], Fri

**Why it holds.** Friday's export repeats each order's booked amount on every payment row, so an order paid in two instalments carries its order_amount twice, and a plain sum or pivot doubles it. Flagging the first row of each order, with =IF(COUNTIF($A$2:A2,A2)=1,1,0) filled down, and summing order_amount over the flagged rows with SUMIFS counts each order once: in Friday's chapter 2 that took the raw export's Rs 39.41 crore back to the warehouse's Rs 19.84 crore. The unused technique beside it in the match table, Remove Duplicates across every column, removes only identical rows: it dropped the 50 gateway copies, and an instalment order's two rows differ in paid_amount, so 1,400 rows stayed and the total still doubled.

**In the interview.** The fix names the grain: one row per order, counted once, before any total is taken.

Anchor: A pivot's total disagrees with the warehouse; where do you look first?

**Q24** **Key C (a labelled input cell feeding a scenario line)** Match the following, Medium, [D], Fri

**Why it holds.** An assumption goes in an input cell, which Friday's workbook colours yellow as the one kind of cell a director may change, beside the actual and feeding a labelled scenario line, so the sheet recalculates in the room and the source stays tied to the warehouse. In Friday's second case, the same Rs 5,00,000 typed over the quarter's cell read "down 14.6 percent" against Finance's 29.4, until Monday's refresh wiped it without a trace.

**In the interview.** I answer the director's question as a labelled scenario beside the actual, refuse the edit to the source, and keep a drift check that ties the sheet to the warehouse on every refresh.

Anchor: Two directors change assumptions in the room and the sheet recalculates differently for each; what did you get right and what do you fix?

**Q25** **Key The sheet reports -0.5 against 1.5, a gap of 2 points; B8 stops at row 5, and over all six countries both columns average 1.5, so the gap disappears.** Applied maths, Hard, [F], Fri

**Why it holds.** B8 averages rows 2 to 5 alone, (-1.0 + 1.0 - 3.0 + 1.0) / 4 = -0.5, while C8 covers all six rows, 9.0 / 6 = 1.5, so the sheet reports growth 2 points lower in high-debt years. Over all six countries the high-debt column averages (-1.0 + 1.0 - 3.0 + 1.0 + 6.0 + 5.0) / 6 = 1.5, the same as the other years, so the gap disappears. In the paper the averages covered lines 30 to 44 of the sheet instead of lines 30 to 49 (footnote 5), and the error, compounded with other errors, took 0.3 percentage points off the published average for the highest debt group, overstated the lowest group by 0.1 and understated the second by 0.2; the replicators' corrected average for the highest group, 2.2 percent against the published -0.1, also corrects the excluded data and the weighting, so the range alone is not the whole gap. The figures are from PERI working paper 322, April 2013, page 7, read on 30 September 2026. The check that catches a range such as B2:B5 is a count of the rows each range covers against the rows in the table, run on every refresh.

**In the interview.** Every total in a workbook ties to a control total from the source on each refresh, and every range is checked against the rows it should cover, because a range that stops short raises no error.

Anchor: Your pivot shows a different total from the warehouse; where do you look first?

**Q26** **Key Column D halves every change, 0.130 against 0.261, so the value at risk should read 140 million dollars, 20 million over the limit.** Applied maths, Hard, [D], Fri

**Why it holds.** Column D reads 0.60 / 4.60 = 0.130, -0.60 / 5.40 = -0.111 and 0.30 / 3.30 = 0.091; divided by the average, as the documentation defines it, the changes are 0.261, -0.222 and 0.182. The sum of the two rates is twice their average, so every change, rising or falling, comes out at half its size, and a value at risk that moves with the changes is half what it should be: 70 million dollars should read 140 million, 20 million over the 120 million limit. The task force wrote that the error "likely had the effect of muting volatility by a factor of two and of lowering the VaR". A second way to the same number, by hand on one row or in a second tool, catches it, as in Thursday's chapter 4, where plain Python, SQL and pandas agreed only once they shared one definition, and Friday's notes met the same report's finding, a spreadsheet that "divided by their sum instead of their average". The quotations are from the Report of JPMorgan Chase and Co. Management Task Force Regarding 2012 CIO Losses, 16 January 2013, pages 123 and 128, read on 30 September 2026, and the value at risk and the limit here are illustrative.

**In the interview.** A number that decides risk gets a second, independent calculation against its written definition before it is trusted, because a formula can be wrong in every row at once.

Anchor: Same question, three tools: how do you choose, and defend one choice?

### Q27 Key 2.00 dollars, about 8.5 percent of the 23.50 dollars charged

Applied maths, Hard, [F], Fri

**Why it holds.** The commission charged is 25 percent of the gross fare on every trip, 7.50, 5.00 and 11.00. The fares after taxes and fees are 27.60, 18.40 and 40.00 dollars, so the commission due is 6.90, 4.60 and 10.00: 0.60 + 0.40 + 1.00 = 2.00 dollars over, which is also 25 percent of the 8.00 dollars of taxes and fees. Against the 23.50 dollars charged that is 8.5 percent, and every trip that carries taxes and fees is overcharged the same way, which is why a percentage's base is part of its definition. CBS News reported on 24 May 2017 that Uber had calculated its driver commissions "based on gross fares, before any taxes and fees were deducted" (read on 30 September 2026 and again on 1 October 2026), and the fares and the rate here are illustrative.

**In the interview.** A percentage is defined only with its base, so before any rate is applied I write down what it is a percentage of.

Anchor: How do you present one number so it is not misread?

### Q28 Key C

One correct option, Hard, [F], Thu

**Why it holds.** The labels hold T3 and T5 twice, so the merge writes one row per matching label and returns 7 rows. T3 and T5 are both tickets the model got wrong, so the mean counts 3 correct rows of 7, 0.43, against 3 correct tickets of 5, 0.6: the model fails the 0.5 bar on a score that counts its two misses twice. It is the fan-out of Tuesday's payments and Thursday's exposure feed, and the checks are the same: rows before and after, or validate='one_to_one'.

**A.** 0.6 is the accuracy on the five tickets, which the code prints only if the labels hold one row per ticket; a merge repeats a left row once for each matching right row.

**B.** 0.71 counts the repeated rows as correct, but they repeat T3 and T5, which the model got wrong, so they pull the mean down.

**D.** A MergeError would need a merge that refuses repeated keys, and frames of different lengths merge without complaint; pandas raises it only when validate is set, and without it writes one row per matching label and prints a number.

**In the interview.** An evaluation metric is a mean over one row per test case, so I size the join before scoring and validate that the labels are unique per case.

Anchor: Your LEFT join grew the row count and revenue doubled; name the cause and the check.

### Q29 Key A

One correct option, Hard, [S], Wed

**Why it holds.** Partitioned by model and ordered by finished_on descending, then run_id descending, ROW_NUMBER puts run 11 first for bot-b, the later of its two runs of 9 September, at 0.74, and run 9 first for bot-a, at 0.78, so bot-a stays, every time the query runs. ROW_NUMBER breaks a tie by whatever else its ORDER BY names, as Wednesday's chapter 3 put it, so the tiebreaker has to follow the business's own definition of latest. run_id is a number, so run 11 sorts above run 10.

**B.** Ascending run_id picks run 10, the earlier of the two runs of 9 September, and promotes bot-b on a run the rule says is not its latest.

**C.** Each max() is taken on its own, so bot-b shows its best score beside its latest date, a pairing no single run produced.

**D.** RANK gives runs 10 and 11 the same rank 1, so bot-b comes back as two rows and the query names no latest run; reading 0.86 off the first of them promotes bot-b on a run the rule says is not its latest.

**In the interview.** The latest row per key is ROW_NUMBER partitioned by the key, ordered newest first with a tiebreaker that makes the order unique and follows the business's own definition of latest, kept at 1 in an outer query.

Anchor: RANK, DENSE_RANK and ROW_NUMBER on a tie.

**Q30** **Key D** One correct option, Hard, [S], Mon

**Why it holds.** Monday's run order is FROM, WHERE, GROUP BY, HAVING, SELECT, ORDER BY, then LIMIT, and because SELECT runs after GROUP BY it can name only the grouping column and aggregates, here model and count(*). WHERE runs first and keeps the five low ratings: three for bot-a, one for bot-b and one for bot-c. GROUP BY forms its groups from those rows alone, so count(*) counts low ratings, and HAVING count(*) >= 3 keeps bot-a with 3. The lead asked for the models with at least three replies in all, bot-a with 4 and bot-b with 3, so bot-b goes unretrained because the HAVING tests the wrong count. With no WHERE, counting every reply in HAVING and the low ones with count(*) FILTER (WHERE rating <= 2) returns bot-a 3 and bot-b 1.

**A.** bot-a and bot-b would need HAVING to judge all of each model's replies, but HAVING runs after WHERE, so it sees only the low ratings, and bot-b's one fails the test.

**B.** Counting all of each model's replies treats WHERE as if it had never run; count(*) counts the rows that survive WHERE.

**C.** Listing all three models ignores HAVING: bot-b and bot-c each have one low rating, and HAVING keeps only groups of three or more.

**In the interview.** WHERE keeps or drops single rows before any group exists; HAVING keeps or drops whole groups after the aggregate is computed, so a HAVING that follows a WHERE counts only the rows WHERE kept.

Anchor: WHERE against HAVING, one sentence each.

**Q31** **Key B** One correct option, Hard, [F], Tue

**Why it holds.** The handoff log holds a row whose conversation_id is NULL. For c1, NOT IN asks c1 <> 'c2' AND c1 <> 'c4' AND c1 <> NULL; the last comparison is unknown, so the whole condition is unknown and WHERE drops the row. Every conversation fails the same way, so the count is 0, and the lead cuts the budget of an assistant that resolved two of four, half. NOT EXISTS, or a LEFT JOIN kept where the handoff key IS NULL, returns 2; Tuesday's chapter 4 named NOT IN as the anti-join to avoid for this reason.

**A.** 2 is the count NOT EXISTS returns; NOT IN against a list that holds a NULL can never be true, so it returns none.

**C.** The NULL sits in the subquery's list, never in the outer table, so it cannot add a conversation.

**D.** Postgres raises nothing: the comparison with NULL is unknown and the rows drop without a warning, so the review gets a wrong figure.

**In the interview.** Rows with no match are an anti-join, which I write as NOT EXISTS or as a LEFT JOIN kept where the right key IS NULL, never as NOT IN against a column that can hold a NULL.

Anchor: How do you find orders with no payment?

**Q32** **Key A** One correct option, Hard, [F], Wed

**Why it holds.** With an ORDER BY and no frame, the window runs from the first row to the current row and its peers, the rows that tie on the ORDER BY. Calls 2 and 3 share 22 September, so both read the day's closing figure, 1,200, and call 2 is routed although the total stood at 700 after it, as the twelve orders of 22 July all read the day's close in Wednesday's chapter 5. Adding call_id to the window's ORDER BY gives each call its own step and routes call 3.

**B.** Call 3 would be the first only if each row took its own step, 400, 700, 1,200, which needs a unique ORDER BY such as day then call_id; with the date alone, calls 2 and 3 are peers and both read 1,200.

**C.** Adding up only the days before each row's own gives 0, 400, 400, 1,200 and routes call 4; the default frame includes the current row and its peers, and a frame that stopped earlier would be written out.

**D.** No call would be routed only if the window were partitioned by day; with ORDER BY day and no PARTITION BY, the total runs across the days and reaches 1,200 on 22 September.

**In the interview.** A running total is deterministic only when its ORDER BY is unique; rows that tie are peers and share one cumulative figure, so I add a tiebreaker such as the id before anyone acts on it.

Anchor: RANK, DENSE_RANK and ROW_NUMBER on a tie.

**Q33** **Key D** One correct option, Medium, [F], Mon

**Why it holds.** A distinct count does not add across days: U1 used the assistant on four days, so U1 is in four daily counts and once in the week's. The week's figure is its own COUNT(DISTINCT user_id) over the week's rows, 6, never the sum of the days, 16. Monday's chapter 5 met the same rule on Kalpa's book: 244 buyers in April to June and 227 in July to September are 301 customers across both quarters, never 471.

**A.** 16 adds up the seven daily counts; each count is distinct within its day, and across days the same users repeat, so the sum counts them again.

**B.** The average of the daily counts, 2.3, answers how many use the assistant on a typical day, a different question from the week's reach.

**C.** 3 is the busiest day's count, which says how many users the busiest day brought, and the ask was the week's active users.

**In the interview.** COUNT(DISTINCT ...) does not add across groups; a period's total needs its own distinct count over the whole period.

Anchor: How do you present one number so it is not misread?

### For the tally

- [S] (7): Q3, Q9, Q13, Q14, Q19, Q29, Q30
- [F] (15): Q1, Q2, Q4, Q7, Q15, Q17, Q21, Q22, Q23, Q25, Q27, Q28, Q31, Q32, Q33
- [SV] (0): none
- [D] (11): Q5, Q6, Q8, Q10, Q11, Q12, Q16, Q18, Q20, Q24, Q26
- Easy (2): Q2, Q3
- Medium (10): Q4, Q5, Q10, Q13, Q14, Q21, Q22, Q23, Q24, Q33
- Hard (21): Q1, Q6, Q7, Q8, Q9, Q11, Q12, Q15, Q16, Q17, Q18, Q19, Q20, Q25, Q26, Q27, Q28, Q29, Q30, Q31, Q32
- Part 1, Can the warehouse give Anand Monday numbers that his analyst can audit line by line? (6): Q1, Q2, Q3, Q4, Q5, Q6
- Part 2, Can Anand sign what Kalpa collected against what it booked, with nothing lost or counted twice? (5): Q7, Q8, Q9, Q10, Q11
- Part 3, Which members should Marketing protect and call, and did July to September keep pace with the plan? (5): Q12, Q13, Q14, Q15, Q16
- Part 4, What must Monday's customer table check before the growth team acts on it? (4): Q17, Q18, Q19, Q20
- Part 5, Which numbers in Monday's workbook can a director trust, and where did three public cases go wrong? (7): Q21, Q22, Q23, Q24, Q25, Q26, Q27
- Part 6, Can a food-delivery company's AI team trust the numbers behind its model and budget decisions? (6): Q28, Q29, Q30, Q31, Q32, Q33

### New items waiting for the tracker

These items come from the week's source file, not the tracker. Accept one by adding it to the tracker's Saturday papers tab.

- Q1 (Applied maths, Hard, [F]): Anand's analyst fills the missing leaf with SELECT quarter, count(*) / count(DISTINCT customer_id) FROM rp_orders GROUP BY quarter, where rp_orders holds the Retail-Plus orders above, and drafts a note to the head of Retail-Plus from what it returns. What does the query print for each quarter, and what should the note say orders per member did from the first quarter to the second? Show the working.
- Q4 (One correct option, Medium, [F]): Retail-Plus has 120 members on its books, and from July to September 76 of them ordered, spending Rs 4,13,380 between them. The head of Retail-Plus asks for the average spend across all 120 members on the books, whether they ordered or not, and the analyst runs the query above. What does it print?
- Q5 (One correct option, Medium, [D]): On Monday afternoon the head of Retail-Plus asks the analyst for a chart of orders per member, week by week across both quarters, to see how the figure moved before anyone decides what to report. With Anand's rule in mind, where should that work happen?
- Q6 (One correct option, Hard, [D]): In 2016 Facebook told advertisers that it had defined the average duration of video viewed as the total time spent watching a video divided by the number of people

who played it, and had calculated it by dividing by only the people who watched for three seconds or more (TechCrunch, 2016). The query above rebuilds both on five illustrative viewers. By how much does the calculated average overstate the defined one?

- Q7 (Scenario set, Hard, [F]): Before the report goes to Anand, the analyst sizes the join by hand. What will the query above return?
- Q8 (Order the steps, Hard, [D]): Five of these six steps make Anand's collected-revenue report, and one would spoil it. Leave that step out, and write the other five in the order they must run.
- Q9 (One correct option, Hard, [S]): Anand wants every order of the quarter beside what was collected on it within the quarter, so the analyst adds a date filter on the payments. Before the report goes to him, the analyst runs the query above to count its rows and their booked value. What does it return?
- Q10 (One correct option, Medium, [D]): It is the end of the reporting day, and Anand expects the report tonight. His rule for a check that fails this late: booked always leaves, because it ties to the orders table alone; a collected figure that does not reconcile never leaves, and an open line goes with the report, naming the failed check, what it means and when it will close. No order on this report was paid short, so a channel's collected reconciles when its gap, booked less collected, equals its unpaid orders' booked value. What do you send?
- Q11 (More than one correct, Hard, [D]): In October 2020 Public Health England found that 15,841 positive COVID-19 cases from 25 September to 2 October had been left out of the daily figures, because some files had exceeded the maximum size the load could take (GOV.UK, 2020). The labs' CSV files had been converted to the older .xls format, and records past its limit were "simply left off and not counted when imported" (The Register, 2020). Which of these checks, run on every file, would have stopped Lab B's file on the day in the table? Mark every correct option.
- Q12 (One correct option, Hard, [D]): Retail-Plus had 76 members who ordered from July to September. Sorted by their revenue for the quarter, highest first, the top 45 each spent a different amount, all above Rs 3,600, and places 46 to 53 are in the table. The head of Retail-Plus set the rule for the protect list: "Ties matter. If two members spent the same, I want them ranked the same, and I want to know how many made the top fifty, not forty-nine because of a tie." The list keeps every member who spent at least as much as the fiftieth member in the table's order, and nobody who spent less. Which ranking gives that list, and how many members does it keep?
- Q14 (One correct option, Medium, [S]): Marketing wants each Retail-Plus member's spend beside the member's share of the tier's spend for July to September. The table member_step holds one row per member who ordered, with the columns customer_id and spend. Which query gives each member's share?
- Q15 (One correct option, Hard, [F]): The falling flag reads member_month, which sums each member's orders by calendar month. It takes prev1 = LAG(spend, 1) and prev2 = LAG(spend, 2), each partitioned by member and ordered by month, and flags a member whose September spend is below prev1 while prev1 is below prev2. Marketing will call every member the flag names and say that their spend fell in both August and September. How many calls does Marketing make, and how many of them say something untrue?
- Q16 (One correct option, Hard, [D]): The quarter closed at Rs 9,84,00,000 against a plan of Rs 9,83,99,990. Meera's chief of staff wants one sentence for Monday's front page on how it got there, and the figures above are all there is. Which sentence do they support?
- Q17 (Scenario set, Hard, [F]): How many rows does the merged table hold, and what should Monday's refresh carry before the pivot is built?
- Q18 (Scenario set, Hard, [D]): The growth team's rule is one exposure per customer, the first by date, and the sale's report credits the sale with every order a customer places from that exposure on. Which exposure does the step above keep for C-0001, and what does the report make of C-0001's orders?
- Q19 (One correct option, Hard, [S]): The head of Retail-Plus wants one row per member and one column per month, to read who is drifting, and a long copy of the same view for a trend chart. What does the code above print?
- Q20 (One correct option, Hard, [D]): Gene symbols are the short names genes go by in papers and databases. Ziemann and colleagues found that Excel's default settings turn some symbols into dates, and that about a fifth of genomics papers with Excel gene lists carried such errors (Genome Biology, 2016). The code above merges a lab's list, after a trip through Excel, onto the lab's reference table of panels, the sets of genes it tests together. What does it print?
- Q21 (Match the following, Medium, [F]): Find any member by id, and print a plain message for an unknown id

- Q22 (Match the following, Medium, [F]): A figure under the protect list that adds the spend of only the members a filter to Mumbai leaves on screen
- Q23 (Match the following, Medium, [F]): Booked revenue by segment from Friday's export, in which an invoice paid in two instalments appears twice
- Q24 (Match the following, Medium, [D]): Let a director try Rs 5,00,000 for Retail-Plus in July to September, with the warehouse's figures untouched
- Q25 (Applied maths, Hard, [F]): In 2013 Herndon, Ash and Pollin rebuilt an influential 2010 paper by Reinhart and Rogoff from the authors' own working spreadsheet, and found that the paper's average growth for countries with public debt over 90 percent of GDP, the value of everything a country produces in a year, published as -0.1 percent, was 2.2 percent once a coding error in the spreadsheet, the exclusion of some available data and an unusual weighting of the averages were corrected (PERI working paper 322, 2013). The illustrative sheet above goes to a review today. What does it report for the two columns, what do the six countries' figures give, and does its comparison hold? Show the working.
- Q26 (Applied maths, Hard, [D]): JPMorgan's task force on the 2012 losses in its Chief Investment Office reported on a value-at-risk model, an estimate of how much a trading book can lose on a bad day, that ran through Excel spreadsheets filled by copying and pasting (JPMorgan Chase, 2013). Suppose the model's documentation defines a day's change as the difference between the new and old rates divided by their average, and that its value at risk moves in proportion to the size of those changes. The sheet above reports a value at risk of 70 million dollars against the desk's limit of 120 million dollars; both figures are illustrative. What value at risk should the sheet have reported, and is the desk inside its limit? Show the working.
- Q27 (Applied maths, Hard, [F]): In 2017 Uber told New York City drivers that it had been taking too much commission from them, and that each affected driver would get a refund of about 900 dollars, interest included (CBS News, 2017). Suppose the drivers' terms set the commission at 25 percent of the fare after taxes and fees. How much commission did the three trips above overcharge the driver, and what share of the commission charged is that? Show the working.
- Q28 (One correct option, Hard, [F]): The team scores a model that sorts support tickets into refund, late and other against labels that an outside firm set by hand, and ships a model only if it scores 0.5 or better. What does the code above print, and what happens to the model?
- Q29 (One correct option, Hard, [S]): The team promotes bot-b over bot-a if bot-b's latest run scores higher than bot-a's latest. Runs are numbered in the order they start, and of two runs that finish on the same day, the one that started later counts as the latest. Which query gives the team the right call, and what is the call?
- Q30 (One correct option, Hard, [S]): Customers rate each reply from 1 to 5. The team lead asks for every model with at least three replies this week, beside how many of its replies were rated 2 or below, and will retrain every model on the list. What does the query above return, and which models are retrained?
- Q31 (One correct option, Hard, [F]): The weekly review reads how many of the four conversations the assistant resolved without a person, and the product lead cuts the assistant's budget if that is under 40 percent. What does the query above return, and what happens to the budget?
- Q32 (One correct option, Hard, [F]): The model's bill is charged per token, so the finance partner routes to the cheaper model every call whose tokens_so_far, read from the query above, is over 1,000. Which is the first call routed?
- Q33 (One correct option, Medium, [F]): The product lead wants the week's active users for the deck. Which number goes on the slide?

### Bank items folded into deeper items

These tracker items are not printed, because a deeper item on the paper tests the same concept.

- Bank 1 (Fill in the blank, Easy, Mon), folded into Q30: The item predicts a query's rows through WHERE, GROUP BY and HAVING in that order, so the clause that filters groups after they form is tested by use.
- Bank 4 (Fill in the blank, Easy, Tue), folded into Q7: Sizing the LEFT JOIN by hand requires knowing that every order stays, with NULLs where no payment matched.
- Bank 5 (Fill in the blank, Medium, Tue), folded into Q31: The anti-join is tested where it breaks, NOT IN against a list that holds a NULL, and the reason names LEFT JOIN with IS NULL as the fix.
- Bank 6 (Fill in the blank, Medium, Wed), folded into Q15: The option of three calls rests on LAG returning NULL for a member with a missing month, and the reason says LAG returns NULL only on a member's first rows.

- Bank 7 (Fill in the blank, Easy, Wed), folded into Q13: Bank 27's key is a rank computed within PARTITION BY segment, so the clause that restarts a window for each segment is the item's answer.
- Bank 8 (Fill in the blank, Medium, Thu), folded into Q17: The guard the item keys is validate='one_to_one', and its reason names the MergeError it raises.
- Bank 9 (Fill in the blank, Easy, Thu), folded into Q19: The code melts the wide view back to one row per member and month, and predicting its row count needs the rule that melt lengthens.
- Bank 10 (True or false, Easy, Mon), folded into Q30: The item's reason gives Monday's run order in full, with SELECT after WHERE, GROUP BY and HAVING, and the query runs in it.
- Bank 11 (True or false, Medium, Mon), folded into Q30: The item's reason says why the group filter sits in HAVING and never in WHERE, and its wrong option a is the reading in which HAVING sees every reply.
- Bank 12 (True or false, Easy, Tue), folded into Q9: The WHERE on the payments table turns the LEFT JOIN into an INNER one and drops the unpaid order without a word, which is the statement bank 12 tested.
- Bank 13 (True or false, Medium, Tue), folded into Q7: The item sizes the fan-out in rows before any rupee is summed, the mechanism by which a join inflates a SUM while every row looks right.
- Bank 14 (True or false, Hard, Wed), folded into Q12: The item's two ties make RANK and DENSE_RANK part company, the only case in which they differ.
- Bank 15 (True or false, Medium, Wed), folded into Q13: Bank 27's key filters the rank in an outer query, which is the reason a window cannot sit in WHERE.
- Bank 16 (True or false, Easy, Thu), folded into Q19: The pivot groups the orders by member and month and returns one cell per group, and predicting that cell is predicting what groupby returns.
- Bank 17 (True or false, Easy, Fri), folded into Q23: The match row is a total taken on an export whose orders repeat, and its reason gives the doubled total a plain sum reports.
- Bank 18 (One correct option, Easy, Mon), folded into Q30: The run order the item's reason gives starts with FROM, the clause bank 18 asks for.
- Bank 19 (One correct option, Medium, Mon), folded into Q30: A syntax error is never an item (CLAUDE.md); the rule behind this one, that SELECT runs after GROUP BY and so sees only grouped columns and aggregates, is in the item's reason.
- Bank 20 (One correct option, Medium, Mon), folded into Q5: The item's reason gives why the reported number lives in the warehouse, rerunning unchanged on the source and read line by line, and the item asks where work that is none of those numbers belongs.
- Bank 21 (One correct option, Easy, Tue), folded into Q7: The item's second count, count(p.payment_id), is the matched rows an INNER JOIN would keep.
- Bank 22 (One correct option, Medium, Tue), folded into Q7: The item sizes exactly how a LEFT JOIN from orders to payments grows past the orders it started from.
- Bank 23 (One correct option, Medium, Tue), folded into Q8: The step the item leaves out drops every order with two payment rows, which is what HAVING count(*) > 1 by order finds; the reason gives the 216 orders it lists against the 28 retries that grouping by order and instalment lists.
- Bank 24 (One correct option, Hard, Tue), folded into Q7: The first check after a doubled total is the row count before and after the join, which this item asks for in numbers.
- Bank 25 (One correct option, Easy, Wed), folded into Q12: The item asks what RANK does after a tie, on the week's real boundary.
- Bank 26 (One correct option, Medium, Wed), folded into Q12: The item asks what DENSE_RANK does after a tie, and its second tie shows the missing gap changing the count.
- Bank 28 (One correct option, Hard, Wed), folded into Q32: The item predicts a running total whose ORDER BY ties, the cause bank 28 names, and its reason gives the tiebreaker.
- Bank 29 (One correct option, Medium, Thu), folded into Q19: The item's reason walks pivot_table as split, apply and combine, with the default mean as the apply step.
- Bank 30 (One correct option, Medium, Thu), folded into Q17: The item sizes a merge that grew because keys repeat on the right, on the week's own feed.
- Bank 31 (One correct option, Medium, Fri), folded into Q21: The match row asks for the lookup that says so when an id is missing, with VLOOKUP's default approximate match as the unused technique beside it.
- Bank 32 (One correct option, Medium, Fri), folded into Q24: The row is the one job bank 32 gives Excel, a director changing an input and watching the number recalculate, with the technique that does it.

- Bank 33 (More than one correct, Medium, Mon), folded into Q30: Predicting the rows needs every statement bank 33 makes about WHERE, HAVING and COUNT(*).
- Bank 34 (More than one correct, Medium, Tue), folded into Q10: The item's table is the validation bank 34 asks for, the gap against the unpaid list channel by channel, and the item goes on to what to do when one channel fails.
- Bank 35 (More than one correct, Hard, Tue), folded into Q7: Option d counts the payments with no order as if they had joined, and its reason says only a FULL OUTER JOIN brings in one side's orphans beside the other's.
- Bank 36 (More than one correct, Hard, Wed), folded into Q13: Top three per segment is one of the questions only a window answers, and the lag and running-total items test the other two.
- Bank 37 (More than one correct, Hard, Wed), folded into Q12: The item applies bank 37's claims about RANK and ROW_NUMBER to the head of Retail-Plus's rule on the week's real tie.
- Bank 38 (More than one correct, Medium, Thu), folded into Q17: The item tests what merge does to rows and the guard that stops a repeated key, and its reason says why a row count is still worth checking.
- Bank 39 (More than one correct, Medium, Fri), folded into Q16: The front-page sentence must carry its period and its comparison; the stem gives the close against plan, and the options differ on the path that led there.
- Bank 40 (Scenario set, Medium, Tue), folded into Q7: The same LEFT JOIN count on the week's own quarter, with instalments as well as repeats.
- Bank 41 (Scenario set, Medium, Tue), folded into Q7: The matched rows are the INNER JOIN's count, the item's second number.
- Bank 42 (Scenario set, Medium, Tue), folded into Q31: The unpaid list is the anti-join; the item tests the pattern that fails and names the ones that work.
- Bank 43 (Scenario set, Medium, Tue), folded into Q8: Dropping each retry's second row before the sum is the item's first step, because a plain SUM counts each retry twice.
- Bank 44 (Scenario set, Medium, Wed), folded into Q12: RANK after a tie, on the week's boundary.
- Bank 45 (Scenario set, Medium, Wed), folded into Q12: DENSE_RANK after two ties, on the week's boundary.
- Bank 46 (Scenario set, Hard, Wed), folded into Q12: The item counts what a DENSE_RANK cut keeps.
- Bank 47 (Scenario set, Hard, Wed), folded into Q12: ROW_NUMBER with a member-id tiebreaker is option b, and its reason names the cost, a member who spent as much as the fiftieth dropped by an id nobody chose.
- Bank 48 (Scenario set, Medium, Thu), folded into Q17: The same merge count on the week's own feed.
- Bank 49 (Scenario set, Medium, Thu), folded into Q17: validate='one_to_one' is the guard the item keys.
- Bank 50 (Scenario set, Medium, Thu), folded into Q17: The item's rows are the pivot's rows, and its reason gives the total they produce, Rs 45,800 over the book.
- Bank 51 (Scenario set, Hard, Thu), folded into Q18: Bank 51's fix, de-duplicating the exposure feed on a stated rule, is the step the item traces, on a feed that arrives out of date order, and the item's answer line gives the sort that applies the rule of one exposure per customer, the first by date.
- Bank 52 (Applied maths, Easy, Mon), folded into Q30: Predicting the query needs the groups GROUP BY forms from the rows WHERE keeps, one per model with a low rating, before HAVING filters them.
- Bank 53 (Applied maths, Medium, Tue), folded into Q8: The Rs 20,750 the retries add to a plain SUM is why the item's first step comes first.
- Bank 54 (Applied maths, Hard, Wed), folded into Q12: RANK ships 51 and ROW_NUMBER 50 on the week's tie at fiftieth.
- Bank 55 (Applied maths, Medium, Wed), folded into Q15: The item runs the falling flag with LAG on four members, two of them with a missing month.
- Bank 56 (Applied maths, Medium, Thu), folded into Q1: Orders per member is the same ratio of two counts, and the trap is the same integer division.
- Bank 57 (Order the steps, Medium, Mon), folded into Q30: Ordering the six clauses is tested by use, since the item's query runs WHERE, GROUP BY and HAVING in the run order, and its reason gives the whole order; printed beside the item, the ordering drill would hand over the step it tests.

- Bank 58 (Order the steps, Medium, Fri), folded into Q5: The item places a piece of work among the warehouse, a notebook and a workbook, and its reason gives the one direction a number travels, from the warehouse through pandas to Excel.
- Bank 19, folded into Q30, option b (accepted): The key was the longest option.
- Bank 22, folded into Q7, option c (accepted): The key was the longest option; the new distractor is the fan-out misread in reverse.
- Bank 23, folded into Q8, option b, d (accepted): The key was the longest option; the distractor is now the full query with the aggregate in WHERE. Option b then ran 38 characters against 73, so it now sorts its distinct list too, and the options run 47 to 73.
- Bank 29, folded into Q19, option a (accepted): The key was the longest option.
- Bank 32, folded into Q24, option c (accepted): The key was the longest option.
- Bank 33, folded into Q30, option c, d; options relabelled, printed a as the tracker's a, b as the tracker's d, c as the tracker's b, d as the tracker's c (accepted): Options ran 22 to 39 characters, with c at 23 and d at 22 against b at 39; c and d now say which clause can compare COUNT(*) with a number, and the options run 34 to 41. Every more-than-one key on the paper held a, and four of the seven were exactly a, b and c. The options are relabelled a, d, b, c, so the wrong option prints at b, the two WHERE statements sit together, and the same three stay correct.
- Bank 34, folded into Q10, option d; options relabelled, printed a as the tracker's d, b as the tracker's a, c as the tracker's b, d as the tracker's c (accepted): The font was a nonsense option, so striking it left a, b and c, which is the whole key; the new distractor is a check that sounds like the other three and cannot catch a fan-out. Every more-than-one key on the paper held a, and four of the seven were exactly a, b and c. The options are relabelled d, a, b, c, so the wrong option prints at a and the same three stay correct.
- Bank 35, folded into Q7, options relabelled, printed a as the tracker's c, b as the tracker's a, c as the tracker's d, d as the tracker's b (accepted): Every more-than-one key on the paper held a, and four of the seven were exactly a, b and c. The options are relabelled c, a, d, b, so the two wrong options print at a and c and the same two stay correct.
- Bank 36, folded into Q13, option b (accepted): Option b ran 25 characters against 60; it now names the quarter its total covers, and the options run 37 to 60.
- Bank 37, folded into Q12, option b (accepted): Option b, part of the key, ran 37 characters against 63; it now says who ties, and the options run 40 to 63.
- Bank 38, folded into Q17, option a (accepted): Option a, part of the key, ran 35 characters against 60; it now says the join is on a key, and the options run 40 to 60.
- Bank 39, folded into Q16, option d (accepted): Cell colour was a nonsense option, so striking it left a, b and c, which is the whole key; the new distractor is the precision a room reaches for when a number is misread.
- Bank 42, folded into Q31, option b, d (accepted): The key was the longest option; the distractor keeps the table-qualified column the key uses. Option d ran 16 characters against 51; it is now the anti-join from the payments side, which finds payments with no order, and the options run 37 to 51.
- Bank 47, folded into Q12, option a (accepted): The key was the longest option.
- Bank 51, folded into Q18, option a (accepted): The key was the longest option.

### The stretch page

**Stretch 1.** The payment's date decides collected's week, where the order's date decides booked's, so an order paid in two instalments adds to collected in whichever weeks its instalments arrive, and an order placed at the end of one week and paid in the next books in one week and collects in the other. On Kalpa's book every paid order was paid within two days of its order, both instalments on the same day, yet 169 of the 648 payment rows on July to September orders land in a different week from their order. A weekly gap between booked and collected is then timing as well as unpaid orders, and the report says so beside the weekly figures.

**Stretch 2.** One row per member with each quarter's spend, kept only where both quarters have an order, then the change member by member and in total, with the count of such members beside it, 60 of the 107 Retail-Plus members who bought on Kalpa's book. Members who ordered in one quarter only answer a different question, about who lapsed and who arrived, so they go in their own line and never into the average change.

**Stretch 3.** Excel changed gene symbols in about a fifth of papers with Excel gene lists, and no reader could protect every copy, so the naming body renamed the genes and fixed the error at the source every reader shares. I would still count, after every merge, the rows against the left table and the keys that found a match, because a key changed on its way in never raises an error.

**Stretch 4.** Rs 19.84 crore is real and covers two quarters, so a director reads it as a quarter that doubled. The page reads: July to September, Rs 9.84 crore, down 1.6 percent on April to June's Rs 10.00 crore.

# Marking grid

The answer sheet with every box of the key marked, for the marker to lay beside each sheet. A starred item is right only when every marked box, and no other, is ticked.

## Lettered items, and true or false

| | A | B | C | D | E |
|---|---|---|---|---|---|
| Q4 | | | X | | |
| Q5 | X | | | | |
| Q6 | | | | X | |
| Q7 | X | | | | |
| Q9 | | X | | | |
| Q10 | | | X | | |
| Q11* | | | X | | X |
| Q12 | | | | X | |
| Q13 | | X | | | |
| Q14 | X | | | | |
| Q15 | | X | | | |
| Q16 | | | X | | |
| Q17 | X | | | | |
| Q18 | | | | X | |
| Q19 | | X | | | |
| Q20 | | X | | | |
| Q28 | | | X | | |
| Q29 | X | | | | |
| Q30 | | | | X | |
| Q31 | | X | | | |
| Q32 | X | | | | |
| Q33 | | | | X | |

* Mark every correct box. A shaded box is a letter the item does not offer.

## The written answers

| | Answer | Key |
|---|---|---|
| Q1 | Final answer | **It prints 2 and 1. The note should say orders per member fell about 22 percent, from 2.36 to 1.84.** |
| Q2 | Letter | **D** |
| Q3 | Letter | **B** |
| Q8 | Letters in order | **E, F, B, C, A** |
| Q21 | Letter | **D** |
| Q22 | Letter | **A** |
| Q23 | Letter | **F** |
| Q24 | Letter | **C** |
| Q25 | Final answer | **The sheet reports -0.5 against 1.5, a gap of 2 points; B8 stops at row 5, and over all six countries both columns average 1.5, so the gap disappears.** |
| Q26 | Final answer | **Column D halves every change, 0.130 against 0.261, so the value at risk should read 140 million dollars, 20 million over the limit.** |
| Q27 | Final answer | **2.00 dollars, about 8.5 percent of the 23.50 dollars charged** |